

AURORA INTELLIGENCE: A ARQUITETA DO PROCESSADOR SOFT-CORE SAPHO

Chrysthofer A. A. Afonso[1] - chrysthofer.afonso@estudante.ufjf.br
Leandro G. Ribeiro[1] - ribeiro.leandro@estudante.ufjf.br
Gustavo B. Libotte[2] - gustavolibotte@iprj.uerj.br
Luciano M. de Andrade Filho[1] - luciano.andrade@ufjf.br
[1]Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN),
Universidade Federal de Juiz de Fora – Juiz de Fora – MG
[2]Instituto Politécnico, Universidade do Estado do Rio de Janeiro – Nova Friburgo – RJ

Resumo. *Projetar um processador soft-core para instrumentação científica é um ciclo de modelagem computacional: o algoritmo é descrito em alto nível, compilado para uma arquitetura gerada sob demanda e validado por simulação antes de chegar ao hardware. Percorrer esse ciclo, porém, exige domínio de linguagens e ferramentas próprias. Este trabalho apresenta a Aurora Intelligence, assistente de inteligência artificial integrada ao ambiente de desenvolvimento AURORA, interface do ecossistema do processador soft-core SAPHO, concebido pelo NIPS-CERN/UFJF. Diferentemente de assistentes genéricas, ela é especializada na linguagem C±, própria do SAPHO, por um system prompt derivado do compilador yanc e por ferramentas de domínio, o que reduz o risco de alucinação sobre as regras do processador. Sua arquitetura é agnóstica ao provedor, opera com modelos de nuvem ou locais e expõe a API do ambiente como um servidor local de Protocolo de Contexto de Modelo (Model Context Protocol, MCP), sob um modelo de permissão. Em estudo de caso, a assistente reconhece de forma autônoma uma redundância semântica no Assembly de um processador de FFT (Fast Fourier Transform) e a elimina dentro do próprio fluxo de compilação, reduzindo o código de 325 para 269 instruções, 17,2% a menos, sem alterar a saída da simulação.*

Palavras-chave: *Aurora Intelligence, Modelos de linguagem, Modelagem e simulação de hardware, Processador soft-core SAPHO, Model Context Protocol*

1. INTRODUÇÃO

Em menos de uma década, a inteligência artificial (IA) generativa passou dos laboratórios ao trabalho técnico cotidiano. Impulsionados pela arquitetura *Transformer* (Vaswani et al., 2017) e pela capacidade de generalizar para tarefas não ensinadas de forma explícita (Brown et al., 2020), os modelos de linguagem de grande porte (*Large Language Models*, LLMs) espalharam-se da redação à análise de dados, e em nenhum campo tão depressa quanto na engenharia de *software*, onde os assistentes de programação já são rotina (Chen et al., 2021; GitHub, 2021).

Na eletrônica, esse impacto alcança o projeto de circuitos digitais, descritos por linguagens próprias (*Hardware Description Languages*, HDLs) como Verilog e VHDL. Já se demonstrou a geração automática de Verilog a partir de linguagem natural (Thakur et al., 2023; Chang et al.,

2023), a adaptação de modelos ao domínio de projeto de *chips* (Liu et al., 2023) e *benchmarks* para a geração de circuitos em nível de transferência de registradores (*Register-Transfer Level*, RTL) (Lu et al., 2024).

Um segundo caminho corre em paralelo: em vez de assistentes isoladas, integrar a IA às ferramentas que o projetista já usa. Editores e ambientes de desenvolvimento passaram a embutir assistentes especializadas no tema que as hospeda, que leem o contexto e os artefatos do usuário e poupam o vaivém entre ferramentas distintas (GitHub, 2021).

É no encontro dessas duas frentes, a IA para HDL e a IA embutida na ferramenta, que este trabalho se situa. O cenário é o da física de partículas. O Centro Europeu de Pesquisas Nucleares (em francês, *Conseil Européen pour la Recherche Nucléaire*, CERN) opera o Grande Colisor de Hádrons (*Large Hadron Collider*, LHC), no qual feixes de prótons se cruzam a 40 MHz (Evans and Bryant, 2008); no detector ATLAS (*A Toroidal LHC ApparatuS*), o calorímetro de telhas (TileCal) reconstrói a energia de jatos hadrônicos (ATLAS Collaboration, 2008; Aad et al., 2010). Processar esses sinais em tempo real e com baixa latência exige *Field-Programmable Gate Arrays* (FPGAs), que aliam paralelismo massivo e eficiência energética (Compton and Hauck, 2002; Ribeiro, 2025). Quando o algoritmo a embarcar é extenso, recorre-se a processadores embarcados (*soft-cores*); e, para não desperdiçar área lógica com arquiteturas fixas, o Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN)[1], da Universidade Federal de Juiz de Fora (UFJF), desenvolveu o SAPHO (*Scalable-Architecture Processor for Hardware Optimization*): um *soft-core* gerado sob demanda, que sintetiza apenas os recursos exigidos por cada aplicação (Kapisch et al., 2026). O SAPHO é programado em C± (lê-se "C mais-menos"), linguagem de médio nível derivada de C, com tipo nativo para números complexos e notação de Dirac para álgebra linear (Chrysthofer Arthur et al., 2026; Dirac, 1958), e seu fluxo de projeto é conduzido pelo ambiente de desenvolvimento integrado (*Integrated Development Environment*, IDE) AURORA (*Advanced Utility Running Optimized Resource Architectures*).

Esse ecossistema já se firma tanto na pesquisa, da geração de fractais (Andrade Filho et al., 2025) à simulação de pulsos do TileCal (Ribeiro, 2025; Luna et al., 2024), quanto no ensino, na cadeira de Dispositivos Lógicos Programáveis (DLP) da graduação e na pós-graduação da UFJF; e é essa maturidade que agora cobra projetos mais rápidos e mais bem otimizados. É daí que parte a Aurora Intelligence: assistente especialista na linguagem C±, integrada à IDE AURORA e conectada à sua interface de programação de aplicações (API). Duas imagens resumem o seu papel. Como regente, reúne e harmoniza, em um único fluxo, as ferramentas, os compiladores e os simuladores de que o projeto depende; como arquiteta, toma parte na concepção da estrutura de que o processador, quase sempre um sistema complexo, emerge. A contribuição é tripla: (i) uma arquitetura de integração agnóstica ao provedor de modelos; (ii) a exposição segura da API do ambiente por um servidor padronizado; e (iii) a especialização da assistente na linguagem e no fluxo proprietários, ancorada nas fontes do compilador.

2. O ECOSSISTEMA SAPHO, AURORA E YANC

Para situar a contribuição, descreve-se brevemente o ambiente no qual a assistente atua. O SAPHO é um processador *soft-core* de arquitetura Harvard e conjunto reduzido de instruções (*Reduced Instruction Set Computer*, RISC), cuja unidade lógica e aritmética (ULA) é combinacional e acumuladora: cada instrução lógico-aritmética opera sobre o acumulador e

[1]Em parceria com o CERN. Mais informações em `https://www.nipscern.com`.

um único operando de memória, executando em um ciclo de *clock* sem quebra do *pipeline* de três estágios. Seu diferencial é a geração de *hardware* sob demanda, pois o montador instancia em Verilog apenas os circuitos efetivamente utilizados pelo programa, o que o aproxima de uma síntese de alto nível (*High-Level Synthesis*, HLS) (Kapisch et al., 2026).

A programação ocorre em C±, subconjunto de C estendido com recursos voltados à física de altas energias: o tipo `comp` para números complexos e a notação de Dirac, em que $\langle a|b\rangle$ denota o produto interno de dois vetores (Chrysthofer Arthur et al., 2026). O cabeçalho de cada processador é configurado por diretivas de *hardware* (largura de palavra, mantissa, expoente, portas de entrada e saída e ganho). A cadeia de compiladores YANC (*Yet Another Compiler*) traduz C± → Assembly → Verilog (executáveis `cmmcomp` e `asmcomp`), enquanto a IDE AURORA, construída em Electron com o editor Monaco (OpenJS Foundation, 2025), orquestra todo o fluxo: edição, compilação, verificação sintática e simulação com o Icarus Verilog ou o Verilator, selecionáveis no próprio ambiente, análise de formas de onda no Surfer (Skarman et al., 2025), visualizador desenvolvido na Universidade de Linköping (LiU), e visualização do RTL no PRISM (*Processor Rendering Interface for Schematic Models*), apoiado em Yosys e *netlistsvg*. Esse fluxo unificado é descrito em detalhe por Chrysthofer Arthur et al. (2026); aqui, ele constitui o substrato sobre o qual a Aurora Intelligence opera.

3. A AURORA INTELLIGENCE: ARQUITETURA E IMPLEMENTAÇÃO

A Aurora Intelligence é a assistente de IA embarcada na IDE AURORA. Esta seção descreve sua arquitetura de integração (Seção 3.1), o modo como a API do ambiente é exposta à assistente (Seção 3.2), os mecanismos que a especializam na linguagem C± (Seção 3.3) e o modelo de segurança que governa suas ações (Seção 3.4).

3.1 Arquitetura agnóstica ao provedor e laço agêntico

A assistente é *agnóstica ao provedor*: a mesma arquitetura opera, por uma interface comum, com modelos de diferentes fornecedores, na nuvem ou localmente. Seu comportamento é *agêntico*: em vez de devolver uma única resposta em texto, ela atua em um laço no qual invoca ferramentas (compilar, simular, ler arquivos), observa os resultados e decide o passo seguinte, até cumprir a tarefa. Esse laço agêntico executa no processo principal da IDE por meio do *Vercel AI SDK* (Vercel, 2025), cuja função de *streaming* intercala texto, chamadas de ferramenta e respectivos resultados, e é limitado a um número máximo de passos por turno, evitando recursões não controladas. Dois transportes convergem para o mesmo formato de eventos (Figura 1): (i) cinco provedores de modelos acessados com chave própria do usuário (*Bring Your Own Key*, BYOK), a saber OpenAI, Anthropic, Google, DeepSeek e Ollama, este último para modelos executados localmente; e (ii) duas assistentes por assinatura, acionadas por interface de linha de comando (*Command-Line Interface*, CLI), Claude Code e Codex. Essa pluralidade permite ao pesquisador escolher entre modelos de fronteira na nuvem e modelos locais, conforme requisitos de custo, latência ou privacidade. A Figura 2 mostra a tela de configuração, na qual o usuário seleciona cada provedor e fornece a chave correspondente.

3.2 A API da AURORA: o servidor MCP e a ponte de ferramentas

A superfície de ação da assistente é um manifesto de 98 ferramentas (20 de leitura e 78 de escrita), organizadas em espaços de nomes que correspondem às funcionalidades da IDE: edição, projeto, compilação, formas de onda e controle de versão (denominado *Dagr*), entre outros. Cada chamada de ferramenta parte do processo principal e é despachada ao *renderer* (o processo que desenha a interface gráfica) por uma ponte assíncrona sobre comunicação entre

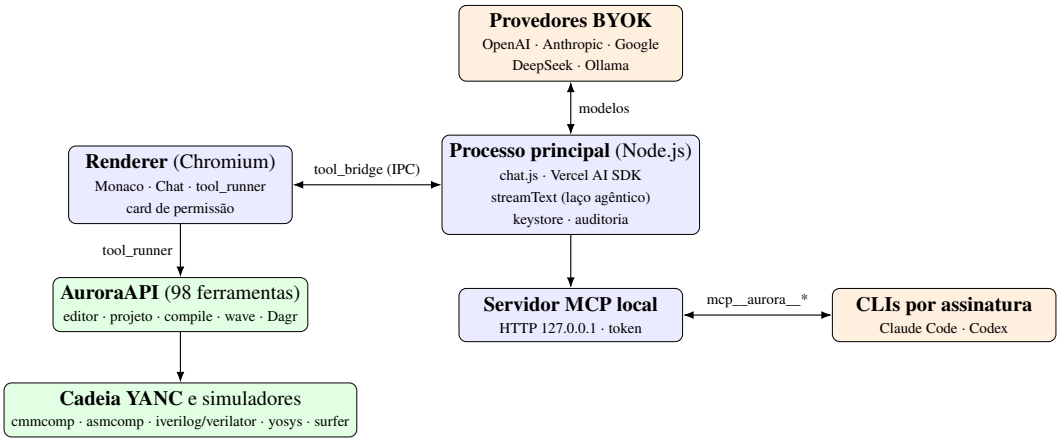


Figura 1: Arquitetura agnóstica ao provedor da Aurora Intelligence. O laço agêntico executa no processo principal; as ações retornam ao *renderer* pela ponte de ferramentas (*tool_bridge*), que invoca a AuroraAPI e, por meio dela, a cadeia YANC. As assistentes por assinatura acessam as mesmas ferramentas pelo servidor MCP local.

Fonte: Autores (2026)

processos (*Inter-Process Communication*, IPC), que executa a ação sobre os artefatos reais e devolve o resultado ao modelo. Falhas nunca interrompem o laço: são devolvidas como resultado de erro, permitindo que o modelo se recupere.

Para que as assistentes por assinatura, que não usam o *Vercel AI SDK*, também alcancem essas ferramentas, o ambiente expõe o mesmo manifesto por meio de um servidor que implementa o *Model Context Protocol* (MCP), um protocolo aberto que padroniza a forma como assistentes de IA se conectam a ferramentas e dados externos (Model Context Protocol, 2024). Trata-se de um servidor HTTP vinculado ao *loopback* (127.0.0.1) em porta efêmera, protegido por verificação de cabeçalho *Host* e por um *token* de sessão de 256 bits, que apresenta cada ferramenta às CLIs no formato `mcp__aurora__<ferramenta>`. Esse servidor é, em sentido estrito, a "API da AURORA": um contrato padronizado e seguro pelo qual qualquer agente de IA compatível com MCP pode inspecionar e operar o ambiente.

3.3 Especialização na linguagem C±

A especialização da assistente, o que a distingue de uma IA genérica, é obtida por dois mecanismos complementares. O primeiro é um *system prompt* extenso e imutável que estabelece sua identidade, vinculada ao NIPS-CERN e ao experimento ATLAS, e codifica de forma verbatim o conhecimento do domínio: os tipos da linguagem C± (`int`, `float`, `comp` e `void`); as diretivas obrigatórias de cabeçalho; as restrições de projeto que, se violadas, causam erro de compilação, como a igualdade estrita entre a largura de palavra e a soma de mantissa, expoente e bit de sinal, e a exigência de que o ganho seja potência de dois; a notação de Dirac; o conjunto de instruções do Assembly do SAPHO (116 *opcodes*, ou códigos de operação); e exemplos reais de código. O segundo mecanismo é um conjunto de ferramentas de domínio, entre elas a listagem de *opcodes* e de diretivas, a análise estrutural de arquivos Assembly e a consulta ao catálogo bilíngue de mensagens do compilador, cujo conteúdo é gerado automaticamente a partir das fontes do *yanc*. Como o conhecimento da assistente é derivado do próprio compilador, e não apenas da memória paramétrica do modelo, reduz-se substancialmente o risco de alucinação sobre a sintaxe e as regras do SAPHO.

Essa especialização ocorre por *condicionamento em contexto* (*in-context*), e não por reajuste dos pesos do modelo (*fine-tuning*). O *system prompt* é montado a partir da gramática do

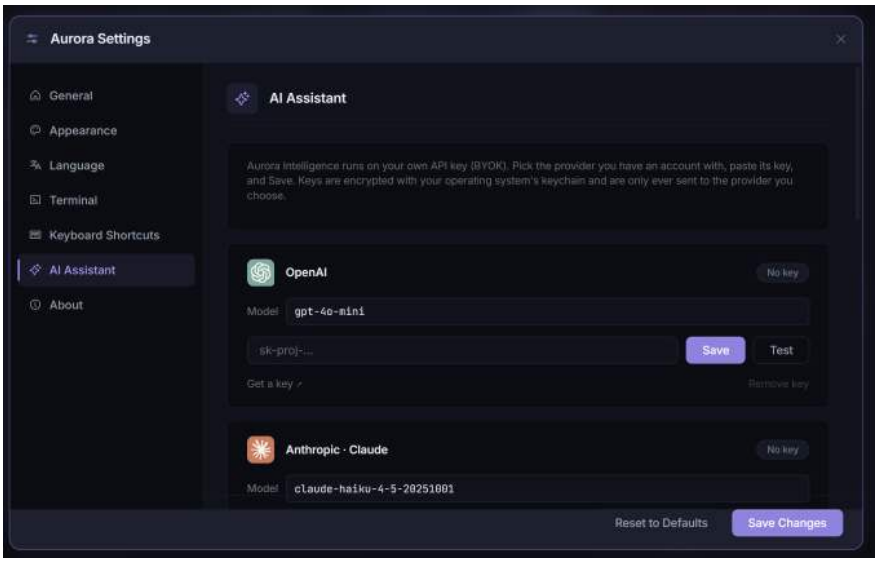


Figura 2: Tela de configuração dos provedores de modelos na AURORA: cada provedor (OpenAI, Anthropic, Google, DeepSeek e Ollama) recebe a chave própria do usuário (BYOK), cifrada pelo cofre do sistema operacional e enviada apenas ao provedor escolhido.
Fonte: Autores (2026)

compilador *yanc* (arquivos `CMMComp.y` e `CMMComp.l`), de exemplos reais de programas C± e do manifesto de ferramentas, e é atualizado sempre que o compilador evolui, o que mantém a assistente fiel à versão corrente da linguagem. Quando o conjunto de instruções do SAPHO cresceu de 112 para 116 *opcodes*, por exemplo, essas quatro novas instruções foram acrescentadas ao *system prompt*, ao contrário do conhecimento fixo embutido nos pesos de um modelo pré-treinado. Por condicionar o modelo sem alterar seus pesos, a especialização permanece portável entre todos os provedores suportados; o *fine-tuning* fica reservado aos trabalhos futuros (Seção 5).

A título de ilustração, a Tabela 1 reúne as diretivas de cabeçalho que declaram a configuração do *hardware*, escritas antes do código do programa.

Tabela 1: Diretivas de cabeçalho de um processador em C±.

Diretiva	**Valor**	**Significado**
`#PRNAME`	`proc_fft`	nome do processador (igual ao do arquivo)
`#NUBITS`	23	largura da palavra (= mantissa + expoente + sinal)
`#NBMANT`	16	bits de mantissa
`#NBEXPO`	6	bits de expoente
`#NDSTAC`	5	profundidade da pilha de dados
`#NUGAIN`	128	constante de ganho do `norm()` (potência de dois)

Fonte: Autores (2026)

3.4 Segurança e governança

Por operar sobre os arquivos do projeto e acionar compiladores, a assistente é submetida a um modelo de permissão com três modos. No modo padrão, ferramentas de leitura executam sem interrupção, ao passo que ferramentas de escrita exibem um *card* de confirmação (Permitir ou Negar) antes de qualquer alteração; os modos alternativos permitem confirmação a cada passo ou autonomia plena. As chaves de API são cifradas em repouso pelo cofre do sistema operacional (*safeStorage*) e nunca atravessam a fronteira de processos, não havendo canal para

sua leitura. Por fim, toda chamada de ferramenta e seu desfecho são registrados em um *log* de auditoria. A Figura 3 ilustra a assistente em operação na IDE, analisando um projeto sob o *card* de confirmação.

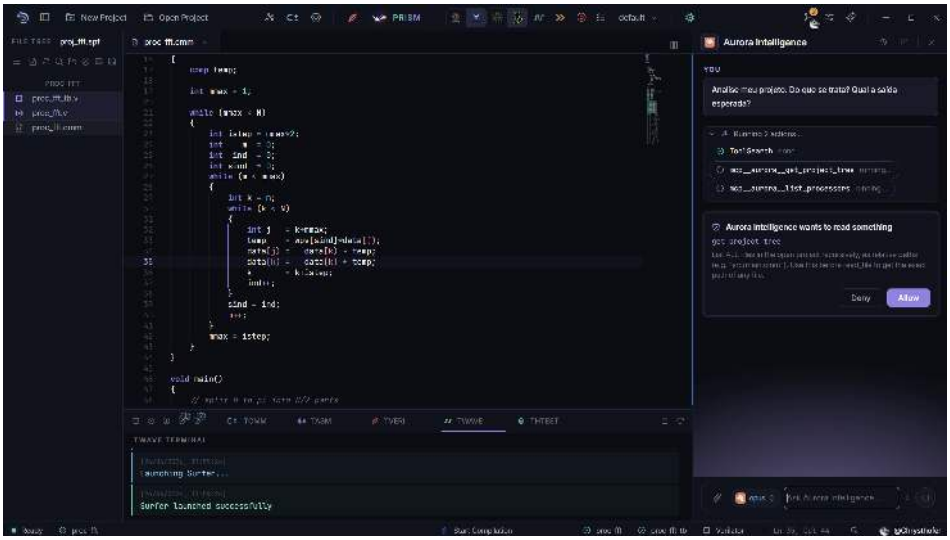


Figura 3: A Aurora Intelligence em operação na IDE AURORA: um pedido em linguagem natural, a execução de ferramentas e o *card* de confirmação (Permitir ou Negar) antes de uma ação.
Fonte: Autores (2026)

4. RESULTADOS E DISCUSSÃO: OTIMIZAÇÃO ASSISTIDA DE ASSEMBLY

Para demonstrar essa orquestração, descreve-se um estudo de caso de otimização do código Assembly de um processador de FFT de oito pontos. O procedimento é o do próprio ciclo de modelagem do ecossistema: o modelo do processador, descrito em C± e traduzido para Verilog pela cadeia YANC, é alterado no nível do Assembly e reavaliado por simulação, de modo que a equivalência funcional entre a versão original e a otimizada seja verificada sobre as formas de onda do modelo simulado, e não por inspeção do código. Com o projeto carregado na IDE, o pedido feito à Aurora Intelligence foi deliberadamente aberto, sem indicar qualquer técnica, para que a própria assistente identificasse as oportunidades:

> *"Dê uma olhada no Assembly gerado para o proc_fft. Você consegue deixá-lo mais eficiente, reduzindo o número de instruções sem alterar o comportamento? Se encontrar oportunidades, aplique as seguras, recompile e simule para confirmar que a saída permanece idêntica, e me mostre quantas instruções havia antes e depois."*

A partir desse pedido, é a própria assistente que traça e conduz o plano: dentre as 98 ferramentas disponíveis, ela decide, a cada passo, quais invocar e em que ordem, sem um roteiro predefinido, encadeando análise, edição, recompilação e simulação até dar a tarefa por concluída.

Primeiro, a assistente executa a sua ferramenta de análise estrutural sobre o Assembly, contabilizando as 325 instruções iniciais e mapeando o uso de cada trecho. Os pares triviais de empilhamento seguido de operação já são fundidos automaticamente pelo compilador *yanc*; a oportunidade encontrada pela assistente é de outra natureza, semântica. No bloco de saída do programa (as 16 chamadas de `fout()`, correspondentes às partes real e imaginária dos oito resultados) havia uma redundância: como `real(data[k])` e `imag(data[k])` são projeções de um mesmo número complexo, o código carregava as duas componentes a cada saída e descartava em seguida a metade não utilizada. A assistente reescreveu o bloco para

carregar apenas a componente necessária, eliminando a carga descartada, como ilustra a comparação a seguir.

Antes (8 instruções)	**Depois** (5 instruções)
`LOD k`	`LOD k`
`ILI data`	`ILI data`
`P_LOD k`	
`ILI data_i`	
`POP`	
`F_MLT 1000.0`	`F_MLT 1000.0`
`F2I`	`F2I`
`OUT 0`	`OUT 0`

Em vermelho, as três instruções eliminadas: carregavam a parte imaginária apenas para descartá-la em seguida.

O código candidato é gravado em uma área isolada (`_aurora_opt/`), sem sobrescrever o arquivo original; um redirecionamento controlado de comando (*command override*) aponta a etapa de montagem para esse arquivo, e a assistente recompila apenas o Assembly, preservando o C± intacto. Só depois de reexecutar a análise estrutural e a simulação, e de aprovado o resultado, o candidato é promovido.

A saída da parte imaginária apresenta a mesma redundância em variante ligeiramente distinta, pois a componente descartada é recuperada por meio de uma variável auxiliar (`SET_P`/`LOD aux_var`), o que permite eliminar ali quatro instruções. Somadas as duas saídas, o padrão economiza sete instruções por índice e, aplicado aos oito índices do bloco, totaliza 56. A economia não depende dos valores em jogo: ainda que a entrada deste exemplo seja real, a saída da transformada é complexa e não nula, e o que se elimina é a leitura de uma componente que aquela chamada não consome. A transformação vale, portanto, para qualquer entrada. Seguindo o fluxo descrito acima, a assistente recompilou apenas o Assembly e reexecutou a simulação no Verilator, que produziu saídas idênticas às do estado original, confirmando a equivalência funcional (Figura 4). O total de instruções caiu de 325 para 269, uma redução de 17,2%, conforme a Tabela 2. Essa redução é estática, isto é, de ocupação da memória de programa. Como as 56 instruções eliminadas são de ciclo único e executam uma só vez, no bloco de saída, a execução também encurta em exatamente 56 ciclos, o que equivale a 4,56% do total simulado. Os dois percentuais diferem porque o primeiro incide sobre as 325 instruções do programa e o segundo, sobre os ciclos executados, mais numerosos por causa dos laços.

Tabela 2: Resultado da otimização assistida do Assembly do `proc_fft`.

Métrica	Valor
Instruções (original, *yanc*)	325
Instruções (otimizada)	269
Redução estática (instruções)	56 (17,2%)
Redução dinâmica (ciclos executados)	56 (4,56%)

Fonte: Autores (2026)

A assistente também examinou o laço de borboletas, núcleo da FFT, e ali identificou uma economia em potencial: fatorar a carga dupla de `data[k]` usada nos termos $-temp$ e $+temp$. Concluiu, porém, que o retorno não compensava o risco, pois a transformação exigiria manipular quatro valores em uma pilha já próxima do limite (`NDSTAC` = 5), podendo alterar o resultado em troca de um ganho modesto. Essa ponderação, a de decidir o que *não* otimizar, evidencia a assistente operando com o discernimento de um projetista, e não como um otimizador cego.

Cabe distinguir os méritos. O desempenho do processador decorre da arquitetura do SAPHO e do seu conjunto de instruções (*Instruction Set Architecture*, ISA); a contribuição da assistente é reconhecer, de forma autônoma e segura, uma redundância de nível semântico que a otimização local do compilador não captura, e eliminá-la dentro do próprio fluxo, sem alterar o C± nem o resultado. É um ajuste que antes dependeria de inspeção manual, como o emprego da diretiva `#USEMAC` relatado por Andrade Filho et al. (2025): a assistente não substitui o projetista, mas amplifica a sua produtividade ao operar o conhecimento da ISA sob supervisão. Registre-se que o estudo de caso desta seção foi conduzido pela própria assistente, sob supervisão dos autores, nos termos da declaração ao final do artigo.

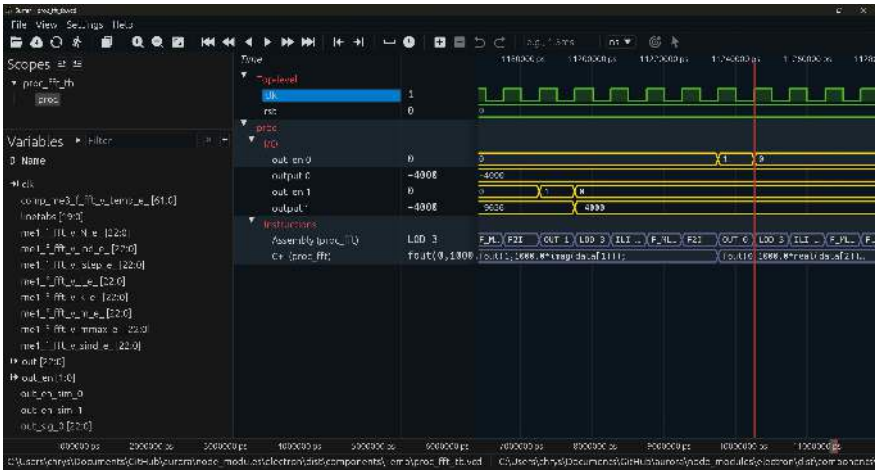


Figura 4: Verificação funcional no Surfer: as formas de onda da simulação do `proc_fft`, com os sinais de saída correlacionados ao Assembly e ao C± do bloco otimizado, confirmando que o resultado permanece idêntico ao da versão original.
Fonte: Autores (2026)

5. LIMITAÇÕES E TRABALHOS FUTUROS

A avaliação tem fronteiras claras. O estudo de caso examina uma única otimização, e não uma bateria sistemática; generalizar a taxa de acerto exige avaliação mais extensa. Como a especialização é obtida em contexto, a qualidade das respostas depende do modelo de base selecionado. A ancoragem nas fontes do compilador reduz, mas não elimina, o risco de alucinação, razão pela qual o modelo de permissão e a confirmação humana seguem indispensáveis. E a equivalência funcional é atestada por simulação sobre as amostras de teste, não por prova formal, de modo que casos de borda não exercitados podem escapar à verificação.

A arquitetura agnóstica ao provedor abre caminho para a principal direção futura: um provedor próprio do NIPS-CERN, com modelos ajustados (*fine-tuned*) às linguagens do ecossistema e servidos localmente, o que reduziria custo e latência e atenderia a requisitos de privacidade, e o uso de recuperação aumentada por geração (RAG) sobre as fontes do *yanc*. O andamento é documentado em `https://www.nipscern.com/projects/ai` (Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN), 2026).

6. CONCLUSÕES

Este trabalho apresentou a Aurora Intelligence, uma assistente de IA integrada à IDE AURORA e especializada na linguagem C± do processador soft-core SAPHO. Mostrou-se que a especialização, obtida por um *system prompt* e por ferramentas de domínio ancoradas

nas fontes do compilador *yanc*, distingue a assistente de uma IA genérica e reduz o risco de alucinação sobre o domínio. Descreveu-se uma arquitetura agnóstica ao provedor, capaz de operar com modelos de nuvem ou locais, e a exposição segura da API por meio de um servidor MCP, governada por modelo de permissão, cofre de chaves e auditoria. O estudo de caso de otimização de Assembly de um processador de FFT evidenciou a assistente na dupla função de arquiteta e regente do fluxo de projeto, ao automatizar, sob supervisão e dentro do fluxo de compilação, uma otimização antes manual. Conclui-se que a especialização de LLMs em linguagens e fluxos proprietários, aliada a uma integração segura, é um caminho efetivo para acelerar e disseminar ferramentas de projeto de *hardware* reconfigurável, tanto em pesquisa quanto no ensino.

Agradecimentos

Os autores agradecem à Universidade Federal de Juiz de Fora (UFJF) e ao Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) pelo suporte institucional e pela infraestrutura, e ao CERN pela parceria que fundamenta as linhas de pesquisa do laboratório. Este trabalho contou com o apoio do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) e da Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG). O presente trabalho foi realizado com apoio da CAPES, Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil, Código de Financiamento 001.

Declaração de uso de inteligência artificial generativa

Este trabalho tem a inteligência artificial generativa como objeto de estudo, e a ferramenta empregada é a própria Aurora Intelligence, o sistema aqui apresentado. O estudo de caso da Seção 4 foi conduzido por ela, na IDE AURORA, a partir de um pedido em linguagem natural deliberadamente aberto: a identificação da redundância, a reescrita do Assembly, a recompilação e a simulação de verificação partiram da assistente, sob o modelo de permissão da Seção 3.4 e com confirmação dos autores a cada ação. A concepção do trabalho, a interpretação dos resultados e as conclusões são de responsabilidade exclusiva dos autores, que revisaram integralmente o manuscrito e por ele respondem.

Referências

Aad, G. et al. (2010). The ATLAS Tile calorimeter: Design, construction and performance. *The European Physical Journal C*, 70:1193–1236.

Andrade Filho, L. M., Chrysthofer Arthur, A. A., Kapisch, E. B., Castro, A. L. B. d., Rimes, S. d. M., and Libotte, G. B. (2025). Implementação de fractal em FPGA usando o processador soft-core SAPHO. In *Anais do XXVIII Encontro Nacional de Modelagem Computacional (ENMC) e XVI Encontro de Ciência e Tecnologia de Materiais (ECTM)*.

ATLAS Collaboration (2008). The ATLAS experiment at the CERN large hadron collider. *Journal of Instrumentation*, 3:S08003.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901. arXiv:2005.14165.

Chang, K., Wang, Y., Ren, H., Wang, M., Liang, S., Han, Y., Li, H., and Li, X. (2023). ChipGPT: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019*.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chrysthofer Arthur, A. A., Torres, G. A., Santos, P. H. S. R., and Andrade Filho, L. M. (2026). AURORA: Ambiente de desenvolvimento integrado para o processador soft-core SAPHO. Projeto de código aberto do NIPS-CERN, Universidade Federal de Juiz de Fora (UFJF). Disponível em: `https://www.nipscern.com/projects/aurora`.

Compton, K. and Hauck, S. (2002). Reconfigurable computing: A survey of systems and software. *ACM Computing Surveys*, 34(2):171–210.

Dirac, P. A. M. (1958). *The Principles of Quantum Mechanics*. Oxford University Press, Oxford, 4 edition.

Evans, L. and Bryant, P. (2008). LHC machine. *Journal of Instrumentation*, 3:S08001.

GitHub (2021). GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`. Acessado em: 22 jun. 2026.

Kapisch, E. B., Aguiar, M. S., Andrade Filho, L. M., and Manso Silva, L. R. (2026). SAPHO—scalable-architecture processor for hardware optimization: An FPGA customizable implementation approach. *IEEE Access*, 14:6820–6832.

Liu, M., Ene, T.-D., Kirby, R., Cheng, C., Pinckney, N., Liang, R., Alben, J., et al. (2023). ChipNeMo: Domain-adapted LLMs for chip design. *arXiv preprint arXiv:2311.00176*.

Lu, Y., Liu, S., Zhang, Q., and Xie, Z. (2024). RTLLM: An open-source benchmark for design RTL generation with large language model. In *Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*.

Luna, F. C., Dias, U. F., Lisboa, P. H. B., Paschoalin, T. C. A., Quirino, T. M., and Andrade Filho, L. M. (2024). Real-time FPGA-based simulator for the Tile calorimeter readout system in the ATLAS experiment. In *Anais do XXVII Encontro Nacional de Modelagem Computacional (ENMC) e XV Encontro de Ciência e Tecnologia de Materiais (ECTM)*, Ilhéus, BA, Brasil.

Model Context Protocol (2024). Model Context Protocol specification. `https://modelcontextprotocol.io`. Acessado em: 22 jun. 2026.

Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) (2026). Aurora Intelligence: Projeto de IA do NIPS-CERN. `https://www.nipscern.com/projects/ai`. Acessado em: 22 jun. 2026.

OpenJS Foundation (2025). Electron: Build cross-platform desktop apps with JavaScript, HTML and CSS. `https://www.electronjs.org`. Acessado em: 22 jun. 2026.

Ribeiro, L. d. G. (2025). Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA. Dissertação (mestrado em engenharia elétrica), Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil.

Skarman, F., Klemmer, L., Große, D., Gustafsson, O., and Laeufer, K. (2025). Surfer: An extensible waveform viewer. In *Computer Aided Verification (CAV 2025), Part IV*, volume 15934 of *Lecture Notes in Computer Science*, pages 392–404. Springer.

Thakur, S., Ahmad, B., Pearce, H., Tan, B., Dolan-Gavitt, B., Karri, R., and Garg, S. (2023). VeriGen: A large language model for Verilog code generation. *arXiv preprint arXiv:2308.00708*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. arXiv:1706.03762.

Vercel (2025). AI SDK by Vercel. `https://ai-sdk.dev`. Acessado em: 22 jun. 2026.

APPENDIX A: ENGLISH TITLE, ABSTRACT AND KEYWORDS

AURORA INTELLIGENCE: THE ARCHITECT OF THE SAPHO SOFT-CORE PROCESSOR

Abstract. *Designing a soft-core processor for scientific instrumentation is a computational modeling cycle: the algorithm is described at a high level, compiled to an architecture generated on demand, and validated by simulation before reaching the hardware. Traversing it, however, demands command of proprietary languages and tools. This work presents Aurora Intelligence, an assistant integrated into the AURORA development environment, the interface of the SAPHO soft-core processor ecosystem, conceived by NIPS-CERN/UFJF. Unlike generic assistants, it is specialized in C±, SAPHO's own language, through a system prompt derived from the yanc compiler and a set of domain tools, which reduces the risk of hallucination about the processor's rules. Its architecture is provider-agnostic, operates with cloud or local models, and exposes the environment's API as a local Model Context Protocol (MCP) server under a permission model. In a case study, the assistant autonomously recognizes and eliminates a semantic redundancy in the Assembly code of an FFT processor, reducing it from 325 to 269 instructions, 17.2% fewer, without altering the simulation output.*

Keywords: *Aurora Intelligence, Language models, Hardware modeling and simulation, SAPHO soft-core processor, Model Context Protocol*